A Mechanized Tensor Core for TTIR-to-TTAdapter Lowering

Anonymous author Author: Please enter affiliation as second parameter of the author macro

Abstract

Tensor compiler transformations often look elementary while depending on shape conditions, finite-width arithmetic, and initialization conventions. We present a source-audited Lean formalization of two conversion patterns in an Ascend Triton compiler: splat and range construction. The source language contains rank-one i32 tensors, inputs, splats, bounded ranges, and pointwise addition. The target language separates index generation, fill, insertion into an arbitrarily initialized tensor, and addition. A shape-indexed lowering selects insertion for singleton splats and fill otherwise; range lowering either returns generated indices or adds a filled signed offset. We prove equality of source and target tensor values for every expression and environment, and derive independence from the initial contents assigned to empty tensors. Ten checked examples exercise signed starts, arithmetic wraparound, and two incorrect hypothetical translations. The development uses exact 32-bit words and includes a principal-theorem axiom audit. Its compiler correspondence is manual: TTAdapter is a mixed-IR pipeline stage, and neither the full stage nor the C++ implementation is verified. The resulting case study makes the local semantic obligations precise while exposing the additional evidence needed for implementation-level correctness.

2012 ACM Subject Classification Software and its engineering → Formal software verification; Software and its engineering → Compilers

Keywords and phrases tensor compilers, Triton, MLIR, Lean, semantic preservation, bit vectors

1 Introduction

Triton expresses tiled computations through an intermediate language and compiler designed for neural-network workloads [5]. In an MLIR-based implementation, a tensor operation can be represented at several levels of abstraction before reaching executable code. MLIR deliberately supports such heterogeneous representations and transformations [2]. For verification, this flexibility raises a basic question: which semantic claim is justified by a proof of a particular lowering pattern?

Consider a scalar splat into a tensor. Filling every lane with the scalar appears straightforward, but a compiler can instead insert the scalar into an empty tensor when the result has exactly one element. The optimization depends on a shape fact: the single write must cover every observable lane. Similarly, a range beginning at zero can be implemented by generated loop indices, whereas a nonzero range requires an offset. The offset is signed, the result is a 32-bit word, and addition is not unrestricted integer addition. Ignoring any of these distinctions can make a superficially plausible account of the translation inaccurate.

We formalize these two patterns from a pinned Ascend Triton checkout. The destination artifact is called `ttadapter` by its Python compiler driver. It is a pipeline stage, not the name of a single MLIR dialect. We therefore introduce an abstract target language for the selected computations rather than claiming to define the semantics of every operation in a TTAdapter module. Our source language is similarly a restricted tensor core, not the complete TTIR language.

The central theorem is a universal equality, not an enumeration of tests. For every source expression, every shape permitted by its constructors, and every environment, evaluating the lowered expression yields the same tensor as evaluating the source expression. Both languages use 32-bit bit vectors. The target explicitly distinguishes a full fill from insertion

at index zero into a tensor with arbitrary initial contents. Consequently, a corollary states that the output of the lowering cannot depend on those contents.

This result has a deliberately limited interpretation. The source and target denotations are manually specified. The translation is a Lean function written to reflect inspected C++ branches, not code extracted from that compiler. We neither execute the compiler nor compare generated MLIR in this study. The contribution is a small, checked semantic specification and an explicit implementation correspondence, not verification of Triton, Ascend, or an end-to-end compilation pipeline.

1.0.0.1 Contributions.

- A shape-indexed, rank-one i32 source and target core that distinguishes signed range attributes, modular arithmetic, and partial initialization.
- Mechanized preservation proofs for both splat branches, both range branches, and their expression-tree compositions with addition.
- A corollary proving independence from arbitrary empty-tensor contents, and ten checked examples including two hypothetical bad-lowering witnesses.
- A revision-specific source audit that separates proven abstractions from unverified compiler, representation, and execution obligations.

We make no claim of a new general compiler-verification technique or priority over all tensor formalizations. The value of the case study is its precise scope, correspondence to concrete implementation branches, and reproducible proof artifact.

2 Compiler Boundary and Motivating Patterns

The audited checkout is the `3rdparty/triton-ascend` subdirectory of a Triton-distributed-ascend repository at revision

`fdecef82c42cade68b4039af35a0e06ac8851536`.

All source anchors in this paper refer to that revision. We use the operation declarations, verifier, conversion patterns, and Python driver as evidence about implementation structure. This is a manual audit; the revision is not a certificate relating executable compiler behavior to Lean.

2.1 The stage is larger than the modeled transformation

The driver registers `stages["ttadapter"]` through `ttir_to_linalg` with named operations enabled. That function invokes several passes, including control-flow and structure transformations, before returning the transformed module text. Temporary and dumped output uses the name `kernel.ttadapter.mlir`. A pure-SIMT route can bypass this stage. Thus, even complete proofs of the selected conversion patterns would not establish preservation by every preceding or following pass.

Our mathematical boundary is a pure tensor computation with fixed inputs. We retain only the value-relevant parts of splat and range construction and provide addition for composition. Source addition is an abstract signless i32 arithmetic operation, not a claimed `tt.add` instruction. Inputs allow the theorem to quantify over arbitrary tensors rather than only closed constant expressions.

2.2 Splat: a full write or a singleton write

The inspected splat converter creates an empty result tensor. If every dimension equals one, it inserts the scalar at all-zero indices. Otherwise it emits a fill. For the rank-one shapes modeled here, the test reduces to whether the length is one. It is incorrect to describe this implementation as unconditionally lowering splat to fill.

For a singleton tensor, insertion and fill have the same value at the sole valid index. For length four, inserting only at index zero does not establish the values of the other three lanes. The formal target makes this distinction observable by assigning arbitrary values to the initially empty tensor. This is a mathematical model of unspecified contents, not a semantics of poison propagation or an assertion that an actual allocation contains particular machine words.

2.3 Range: generated indices and a signed offset

The range converter constructs a `linalg.generic` operation with identity indexing and a parallel iterator. Its body obtains the loop index, casts it to i32, and yields it. With start zero, this tensor is the result. With nonzero start, the converter fills another tensor with the signed start constant and combines it with the index tensor using `arith.addi`.

The declaration of `tt.make_range` explicitly warns that generated accessors can return unsigned values although the attributes represent signed i32. The inspected converter and verifier read the signed attribute through `getStartAttr().getInt()`. Our model consequently encodes the start with `BitVec.ofInt`, not an unchecked conversion to a natural number. An example starting at -2 tests this choice.

The verifier requires a nonempty interval, a rank-one result, a length equal to the difference between endpoints, and i32 elements. We encode a restricted well-formed range directly in the source constructor. We do not model parsing attributes or prove equivalence with the full verifier.

3 A Shape-Indexed Tensor Core

3.1 Words, shapes, and environments

Let $\mathbb{B}_{32}$ be the set of 32-bit words, represented by Lean's `BitVec 32`. Write $\langle z \rangle_{32}$ for conversion of an integer z to its 32-bit representation, and $u \oplus v$ for addition modulo 2^{32}. For a natural number n, define

$$\mathsf{Fin}(n) = \{i \in \mathbb{N} \mid i < n\}, \qquad \mathsf{Tensor}(n) = \mathsf{Fin}(n) \to \mathbb{B}_{32}.$$

The Lean representation is `Fin n âĘŠ BitVec 32`. This is an extensional tensor semantics: two tensors are equal when their values at every valid index are equal. It says nothing about physical layout or memory addresses.

An environment $\rho = (I, U)$ contains two total families:

$$I : \prod_{n:\mathbb{N}} (\mathbb{N} \to \mathsf{Tensor}(n)), \qquad U : \prod_{n:\mathbb{N}} \mathsf{Tensor}(n).$$

$I(n, x)$ supplies input identifier x at shape n, while $U(n)$ supplies the initial contents used by the target's insertion operation. Input identifiers are shape-indexed; the same natural number at a different shape does not impose a consistency condition. The model does not implement SSA names, dominance, or a variable-binding discipline.

The family U chooses one tensor per shape. It does not provide independent fresh choices for each allocation, nor temporal nondeterminism. Our theorem is independence from this explicitly encoded family. Establishing refinement from a more detailed nondeterministic or poison-aware semantics remains a separate obligation.

3.2 The range domain

For a signed start $s \in \mathbb{Z}$ and length $n \in \mathbb{N}$, set

$$V(s,n) \quad \equiv \quad 0 < n \ \wedge\ -2^{31} \le s \ \wedge\ s+n \le 2^{31}-1. \tag{1}$$

The exclusive end is represented by $s+n$, not a separate stored parameter. Since the interval is nonempty, these inequalities also bound the start above and the end below. Both endpoints are representable as signed i32. An exclusive end of 2^{31} is intentionally excluded even if some mathematical lane sequence ending just before it would be representable.

The `makeRange` constructor carries a proof of $V(s,n)$. Range validation is therefore a construction obligation rather than a possible failure of the evaluator. Other constructors may have length zero: their denotations are empty functions. This does not assert that every such shape is legal in the actual TTIR implementation.

3.3 Indexed syntax

For each n, the source syntax S_n has the following constructors:

$$e ::= \mathsf{input}_n(x) \mid \mathsf{splat}_n(v) \mid \mathsf{range}_n(s,h) \mid e_1 +_n e_2, \qquad h : V(s,n).$$

Both operands of $+_n$ belong to S_n. The target syntax T_n is

$$t ::= \mathsf{input}_n(x) \mid \mathsf{fill}_n(v) \mid \mathsf{insert0}_n(v) \mid \mathsf{index}_n \mid t_1 +_n t_2.$$

Here $v \in \mathbb{B}_{32}$. The size indices are part of the inductive Lean types, so ill-shaped additions cannot be constructed. In particular, a function of type $\mathsf{S}_n \to \mathsf{T}_n$ preserves the represented length by construction. This intrinsic property must not be confused with a separate theorem about the real compiler's type converter.

3.4 Source and target denotations

For $i \in \mathsf{Fin}(n)$, source evaluation is defined by

$$\begin{aligned}
[\![\mathsf{input}_n(x)]\!]^{\mathsf{S}}_{\rho}(i) &= I(n,x)(i),\\
[\![\mathsf{splat}_n(v)]\!]^{\mathsf{S}}_{\rho}(i) &= v,\\
[\![\mathsf{range}_n(s,h)]\!]^{\mathsf{S}}_{\rho}(i) &= \langle s\rangle_{32} \oplus \langle i\rangle_{32},\\
[\![e_1 +_n e_2]\!]^{\mathsf{S}}_{\rho}(i) &= [\![e_1]\!]^{\mathsf{S}}_{\rho}(i) \oplus [\![e_2]\!]^{\mathsf{S}}_{\rho}(i).
\end{aligned}$$

The range denotation is modular from the outset. Bounds on the corresponding mathematical integers are proved separately. We have not mechanized a general bridge stating that signed decoding of every computed range word equals $s+i$ under $V(s,n)$. Such a bridge is not needed by the preservation proof, but would strengthen correspondence to a mathematical range specification.

Target evaluation uses the following equations:

$$[\![\mathsf{input}_n(x)]\!]^{\mathsf{T}}_\rho(i) = I(n,x)(i),$$
$$[\![\mathsf{fill}_n(v)]\!]^{\mathsf{T}}_\rho(i) = v,$$
$$[\![\mathsf{insert0}_n(v)]\!]^{\mathsf{T}}_\rho(i) = \begin{cases} v & i = 0, \\ U(n)(i) & i \neq 0, \end{cases}$$
$$[\![\mathsf{index}_n]\!]^{\mathsf{T}}_\rho(i) = \langle i \rangle_{32},$$
$$[\![t_1 +_n t_2]\!]^{\mathsf{T}}_\rho(i) = [\![t_1]\!]^{\mathsf{T}}_\rho(i) \oplus [\![t_2]\!]^{\mathsf{T}}_\rho(i).$$

The index constructor compresses index generation, cast, and yield into one pure operation. It presupposes that the surrounding index representation can express the modeled loop indices; no MLIR data-layout or index-width theorem is provided. The insert0 constructor has a total denotation even at length zero, where there are no valid observations. It is not a semantics of invalid `tensor.insert` operations. The lowering emits it only at length one.

4 Lowering and Semantic Preservation

Define $\mathcal{L} : \mathsf{S}_n \to \mathsf{T}_n$ structurally. Inputs are preserved and addition lowers recursively. The interesting cases are

$$\mathcal{L}(\mathsf{splat}_n(v)) = \begin{cases} \mathsf{insert0}_n(v) & n = 1, \\ \mathsf{fill}_n(v) & n \neq 1, \end{cases} \tag{2}$$

$$\mathcal{L}(\mathsf{range}_n(s,h)) = \begin{cases} \mathsf{index}_n & s = 0, \\ \mathsf{index}_n +_n \mathsf{fill}_n(\langle s \rangle_{32}) & s \neq 0. \end{cases} \tag{3}$$

The range witness h does not affect the generated target expression. It establishes membership in the chosen source domain. In fact, the bit-vector identity underlying the range proof does not require its bounds; the bounds concern admissibility and the intended signed-range reading.

▶ **Lemma 1** (Splat preservation). *For every n, v, ρ, evaluating the right-hand side of Equation (2) yields the constant tensor with value v.*

Proof. By extensionality, fix $i \in \mathsf{Fin}(n)$. If $n = 1$, then $0 \leq i < 1$ implies $i = 0$. Target insertion therefore returns v, not $U(n)(i)$. If $n \neq 1$, the target is a fill and returns v at every valid index. These are exactly the source splat values. The length-zero case in the second branch is vacuous. ◀

This is `lower_splat_correct`. Its singleton argument is not an assumption that empty tensors are zero: it proves that no initial value is observed. This distinction is essential to the negative example in Section 5.

▶ **Lemma 2** (Range preservation). *For every $n, s, h : V(s,n), \rho$, evaluation of the right-hand side of Equation (3) equals $i \mapsto \langle s \rangle_{32} \oplus \langle i \rangle_{32}$.*

Proof. Fix a valid index i. If $s = 0$, the target returns $\langle i \rangle_{32}$ and the source returns $\langle 0 \rangle_{32} \oplus \langle i \rangle_{32}$, equal by the zero identity. Otherwise, the target returns $\langle i \rangle_{32} \oplus \langle s \rangle_{32}$. Commutativity of bit-vector addition gives the source value. Extensionality gives equality of the tensors. ◀

The Lean theorem `lower_range_correct` uses `BitVec.add_comm`. It establishes the equality for finite-width operations; no silent replacement by unbounded addition occurs.

▶ **Theorem 3** (Lowering correctness). *For every n, environment ρ, and expression $e \in \mathsf{S}_n$,*

$$[\![\mathcal{L}(e)]\!]^{\mathsf{T}}_{\rho} = [\![e]\!]^{\mathsf{S}}_{\rho}.$$

Proof. Proceed by structural induction on e. The input case follows from the shared environment lookup. Lemmas 1 and 2 discharge the splat and range cases. For $e_1 +_n e_2$, fix a lane i. The target adds the values of $\mathcal{L}(e_1)$ and $\mathcal{L}(e_2)$ at i. The induction hypotheses replace these with the source values of e_1 and e_2, after which the source addition equation applies. Extensionality finishes the case. ◀

The machine-checked statement is `lowering_correct`. It covers all finite expression trees in the grammar, arbitrary input tensors, and arbitrary U.

The corollary `lowering_lane_correct` specializes the equality to any $i \in \mathsf{Fin}(n)$. There is no execution trace, concurrency relation, or termination claim about the compiler here.

4.1 Independence from initial contents

Source evaluation never consults U. Structural induction proves that environments with equal input families give equal source results; the artifact calls this `source_ignores_empty`. Combining this property with Theorem 3 gives a target property.

▶ **Corollary 4** (Empty-content independence). *If $\rho_1 = (I, U_1)$ and $\rho_2 = (I, U_2)$, then for every $e \in \mathsf{S}_n$,*

$$[\![\mathcal{L}(e)]\!]^{\mathsf{T}}_{\rho_1} = [\![\mathcal{L}(e)]\!]^{\mathsf{T}}_{\rho_2}.$$

Proof. Rewrite each side to its source denotation using Theorem 3, then apply source independence. ◀

This is `lowering_ignores_empty`. It is stronger than checking results with a particular zero-initialized environment, but narrower than proving absence of undefined behavior in generated MLIR. The theorem is about the stated value model and the image of $\mathcal{L}$, not every target term. Indeed, a general insert0 term at length greater than one may depend on U.

4.2 Mathematical range bounds

The artifact separately checks

$$s \le s + i < s + n,$$
$$-2^{31} \le s + i < 2^{31} - 1$$

for a valid range and lane. These are `range_lane_bounds` and `range_lane_signed_bounds`. The first inequality only needs $i \in \mathsf{Fin}(n)$; its range-validity parameter is unused in the proof. The second uses the endpoint bounds. Its strict upper bound follows from the exclusive end, itself bounded above by $2^{31} - 1$. These integer facts are not a substitute for the unproved general signed-decoding bridge identified in Section 3.

5 Mechanization and Checked Examples

The implementation uses Lean 4.19.0 and its bundled `Std` library, with no external library dependencies. Lean supports both the functional definitions and their proofs in one language [1]. The development separates the core definitions, lowering proofs, examples, and an axiom inspection file. It is independent of the neighboring synchronization formalization; no synchronization result is imported to justify tensor lowering.

Table 1 Artifact structure and checked evidence. Paths are relative to the formalization root, one directory above this paper.

File	Role
`TTIRFormal/Core.lean`	Indexed syntax, denotations, two integer-bound theorems.
`TTIRFormal/Lowering.lean`	Lowering, two local preservation results, composition, lane equality, and two independence results.
`TTIRFormal/Examples.lean`	Ten concrete example and counterexample theorems.
`TTIRFormal/Audit.lean`	Axiom dependencies of eight principal results.

5.1 Positive cases

The example environment supplies zero-valued inputs and assigns 99 to every initially empty lane. A length-four range starting at -2 checks signed decoding of its first lane as -2 and its last lane as 1. A zero-start range checks its last lane as 3. A singleton splat checks the insert branch with value 7. Adding a length-four splat of 7 to the negative range checks the final lane as 8 and exercises composition and the fill branch. Finally, adding singleton splats of $2^{32} - 1$ and 1 checks a result of zero, demonstrating that the source arithmetic is modular.

These six positive theorems complement rather than establish the universal result. For example, the wraparound case checks the chosen arithmetic interpretation, while the induction theorem already quantifies over every input word. No benchmark timing or performance comparison is asserted.

5.2 Two hypothetical incorrect translations

The first counterexample replaces the nonzero-start range lowering with bare index generation. At lane zero of the range beginning at -2, the bad target produces $\langle 0 \rangle_{32}$ and the source produces $\langle -2 \rangle_{32}$. Their inequality is checked by `dropped_offset_counterexample`. This is a witness against an alternative mathematical translation, not an observed defect or mutation test run against the C++ compiler.

The second uses singleton insertion for a length-four splat of 7. At lane one, the target returns the initial value 99 while the source returns 7. The inequality is checked by the theorem

`oversized_insert_counterexample`.

It witnesses failure of equality for this environment; it does not say every environment would reveal the defect. If the unwritten contents happened already to be 7, this particular observation would not distinguish the translations. Universal quantification over initial contents is what rules out relying on that accident.

5.3 Domain rejection and coverage

Two further theorems prove that $V(0, 0)$ and $V(2^{31} - 1, 2)$ are false. These are model-domain checks, not executions of the upstream verifier. Together with the six positive cases and two bad-lowering witnesses, they constitute ten checked example theorems. They do not measure coverage of the compiler codebase or prove completeness of the formal grammar.

5.4 Proof validation and assumptions

A clean `lake build` checked the library and examples. The command

Table 2 Source anchors at the audited revision. Long directory prefixes are factored into the key below the table.

Anchor	Inspected fact
A: `TritonOps.td`:427	Splat declaration and tensor result.
A: `TritonOps.td`:854	Rank-one i32 range; signed-accessor warning.
B: `Ops.cpp`:407	Nonempty interval, length, rank, and element verifier.
C: `TritonOpConverter.cpp`:1145	Generated indices and optional start offset.
C: `TritonOpConverter.cpp`:1205	Singleton insertion versus fill.
C: `TritonOpConverter.cpp`:1350	Broadcast collapse and possible match failure.
D: `compiler.py`:324,1592	Stage implementation and registration.

A: `include/triton/Dialect/Triton/IR/`; B: `lib/Dialect/Triton/IR/`;
C: `third_party/ascend/lib/TritonToLinalg/`; D: `third_party/ascend/backend/`.

```
lake env lean TTIRFormal/Audit.lean
```

inspected the assumptions of the principal results. The local lowering results, universal preservation, target independence, and both bad-lowering witnesses report `propext` and `Quot.sound`. The two integer-bound results additionally report `Classical.choice`. These are standard Lean foundations, not new compiler axioms introduced by the artifact.

The proof sources contain no `sorry`, `admit`, custom axiom declarations, or `native_decide` shortcuts. Tactics simplify equations, discharge finite examples, and solve arithmetic obligations; the reported evidence is successful proof checking, not a claim that the specification itself is independently validated. A proof assistant cannot determine whether a manually encoded denotation is the intended semantics of a separately implemented compiler.

6 Source Correspondence and Remaining Obligations

Table 2 records the inspected anchors. It is a correspondence map for audit, not a machine-checked refinement relation. The full revision is recorded in Section 2 and in the accompanying findings.

6.1 What is omitted from the patterns

The range converter records three attributes:

- `tt.from_make_range`;
- `tt.make_range_offset`;
- `tt.make_range_size`.

The model omits this metadata and does not prove that downstream transformations interpret it correctly. The splat converter inserts debug NOP anchors that are likewise omitted. Evaluation models result values, not every generated operation or diagnostic effect.

We inspected broadcast but did not add it to the formal syntax. Its converter computes broadcast and unbroadcast dimensions, asks for collapse reassociation indices, and can fail matching. On success it constructs collapse-shape and broadcast operations. A proof would need to account for these shape helpers and success conditions rather than simply adding an unconditional replication equation. No broadcast theorem is claimed.

6.2 A layered correctness argument

The current evidence occupies only part of a potential end-to-end argument:

1. *Source representation.* Establish that a concrete TTIR fragment is represented by a well-formed source expression with the same observations. This requires type, attribute, operand, and relevant layout correspondence.
2. *Abstract lowering.* Prove preservation by $\mathcal{L}$. This is Theorem 3, completed for the stated grammar.
3. *Implementation correspondence.* Show that the actual converter implements $\mathcal{L}$ under explicit assumptions, including index casts, allocation behavior, and pattern applicability. The present audit is informal evidence for this step, not a proof.
4. *Surrounding passes and execution.* Establish preservation through the remaining passes, bufferization, code generation, and execution. None of these steps is provided by this artifact.

This decomposition explains why a short algebraic preservation proof can be useful without establishing a large systems claim. The small proof isolates exactly where the singleton guard and signed offset matter; the remaining work is to justify that these abstractions describe execution.

6.3 Threats to validity

The principal threat is specification error shared by both denotations. In particular, defining the source range by bit-vector addition makes its relation to the target transparent, but does not independently establish the full meaning of the upstream operation. The separate bounds and concrete signed examples mitigate only part of this issue. A general signed-decoding theorem and a verified representation relation would provide stronger evidence.

The pure, total, shape-indexed representation intentionally eliminates several classes of real compiler behavior: malformed IR, dynamic shapes, SSA control flow, memory effects, masks, pointer arithmetic, aliasing, concurrency, and floating point. It also does not distinguish independent empty allocations or model poison. These omissions limit applicability; they are not conclusions that the omitted behaviors are safe.

Finally, source inspection is revision-specific and does not automatically track future changes. The current artifact has no parser or automated source-to-model validator. Rechecking Lean after a compiler change cannot by itself establish that the source correspondence still holds.

7 Related Work

Triton introduced an intermediate language and compiler for tiled neural-network computations [5]; our artifact is not a formalization of that entire programming model. It studies two patterns in a particular downstream checkout. MLIR provides extensible compiler infrastructure across abstraction levels [2]; our distinction between a mixed-IR stage and a single dialect follows the practical need to identify a narrower semantic boundary within such a pipeline. We do not supply a general framework for composing arbitrary dialect semantics.

CompCert exemplifies machine-checked semantic preservation for a realistic compiler [3]. The present work uses the familiar structure of source and target semantics connected by a translation theorem, but has a far smaller language and lacks a verified connection to the actual C++ transformation. It should therefore not be read as an alternative end-to-end verified compiler.

Alive2 addresses bounded translation validation for LLVM [4]. Our proof is universal over the encoded expression trees rather than validation of individual LLVM transformations, but the implementation boundary is weaker: we do not consume before-and-after compiler IR. The two bad-lowering witnesses are not a translation-validation engine. A future validator for the selected TTIR/target fragment could complement the model, provided parsing, unsupported cases, and the validation semantics were made explicit.

Lean 4 supplies the dependent types, executable functions, and theorem proving infrastructure used here [1]. We exploit shape indices to constrain constructors and use function extensionality to reason about tensors. Our contribution is the source-specific case study, not a new foundational logic or dependent tensor representation. This discussion situates the artifact among compiler proofs and validation approaches; it is not a systematic novelty survey.

8 Conclusion

Two elementary-looking tensor conversions expose useful verification obligations: a singleton write must cover every valid lane, and range construction must retain the signed offset under finite-width arithmetic. The Lean core makes both obligations explicit and proves value preservation under arbitrary input and initial-content families. Its checked counterexamples distinguish plausible incorrect translations, while the axiom audit records the logical foundations of the principal results.

The next steps are not larger claims about the same proof. They are additional evidence: a signed-decoding bridge, a concrete representation relation, a correspondence argument for the actual conversion patterns, and preservation through surrounding passes. The present artifact provides a small verified component and a precise account of what remains outside it.

Artifact Availability

The accompanying `ttir-formalization` directory contains all Lean sources, a pinned toolchain, build configuration, and source-audit findings. From its root, `lake build` checks the library and examples; `lake env lean TTIRFormal/Audit.lean` prints the principal axiom dependencies. The paper resides in its `paper` subdirectory. No public archive DOI or independent artifact-evaluation result is claimed. The artifact does not require Ascend hardware, and no hardware performance or compiler-execution experiment is reported.

Generative AI Disclosure

Generative AI assisted the development of the accompanying formalization and the drafting and editing of this manuscript. The reported mechanized results were checked with Lean. This draft requires author review of the scientific claims, source correspondence, and references before submission. AI assistance is not a substitute for that review or for authorship responsibility.

References

1 Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction—CADE 28*, 2021. doi:10.1007/978-3-030-79876-5_37.

2 Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: A

compiler infrastructure for the end of Moore's law, 2020. arXiv:2002.11054. doi:10.48550/arXiv.2002.11054.

3 Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 2009. doi:10.1145/1538788.1538814.

4 Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. Alive2: Bounded translation validation for LLVM. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2021. doi:10.1145/3453483.3454030.

5 Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019. doi:10.1145/3315508.3329973.